FORMATION « PYTHON EN 24 HEURES » | SÉANCE 05

Listes, tuples et compréhensions

Chapitres 9 et 10 du Tome 1

1 h de cours (présentation Beamer) • 1 h de pratique (notebook Jupyter)

HOUNKPE Mawulikplimi Roland et DJABON Ounimborbitibou

Objectifs de la séance

- Créer, lire et modifier des listes avec index, slices et méthodes
- Parcourir une liste avec for, enumerate et le test d'appartenance in
- Distinguer les tuples, figés une fois pour toutes, et leurs usages
- Écrire des compréhensions de listes simples, filtrées et imbriquées
- Éviter le piège de la copie de liste partagée

Rappel de la séance 04

Les listes sont le premier conteneur du cours : une seule variable pour ranger toute une série, parcourue par les boucles que vous maîtrisez déjà.

- if, elif, else : les décisions en Python et l'indentation structurante
- and, or, not : les connecteurs logiques
- while et for : les deux boucles, break et continue
- range(a, b) compte de a à b moins 1 : la borne de fin est exclue
- Une comparaison produit un bool : c'est le carburant des filtres

Créer et lire une liste

```python
notes = [12, 15, 9, 18]
print(notes[0])      # 12  premier élément
print(notes[-1])     # 18  dernier élément
print(notes[1:3])    # [15, 9]  slice : borne de fin exclue
print(len(notes))    # 4
```

Les index démarrent à 0. Les index négatifs comptent depuis la fin : notes[-1] est le dernier.

Modifier une liste : les méthodes clés

- ‘append(x)‘ : ajoute à la fin
- ‘insert(i, x)‘ : insère à la position i
- ‘remove(x)‘ : retire la première occurrence de x
- ‘pop(i)‘ : retire et **retourne** l’élément à l’index i
- ‘sort()‘ et ‘reverse()‘ : rangent la liste **sur place**
- ‘in‘ : teste l’appartenance, renvoie un bool

```
panier = ["riz", "huile"]
panier.append("sucre")
panier.insert(0, "pain")
panier.remove("huile")
dernier = panier.pop()
print(panier, dernier)
print("riz" in panier)
```

Parcourir : for, enumerate

```
notes = [12, 15, 9]
for n in notes:
    print(n)

for i, n in enumerate(notes):
    print(f"Élément {i} : {n}")
```

Préférez le parcours direct for n in notes à la manipulation d'index. enumerate fournit la position quand elle est vraiment utile.

Le tuple : une liste figée

- Parenthèses au lieu de crochets : (3, 5)
- **Immuables** : impossible de modifier un élément
- Ideal pour un résultat composé : (somme, moyenne)
- Unpacking : ‘a, b = point‘ déplie le tuple en deux variables
- Retourner plusieurs valeurs d’une fonction : c’est un tuple

```
point = (3, 5)
x, y = point          # unpacking
print(x + y)

def stats(notes):
    return (min(notes), max(notes))
mini, maxi = stats([7, 9, 14])
print(mini, maxi)
```

La compréhension de liste : l'idée

- Un motif ultra courant : créer une liste en transformant une autre suite
- Version classique : boucle for + append, trois lignes
- La compréhension tient en **une seule ligne lisible**
- '[expression for element in suite]'
- Python exécute exactement la même chose, mais le code dit l'essentiel en premier

Boucle classique contre compréhension

```python
# Version classique
cubes = []
for k in range(1, 6):
    cubes.append(k ** 3)

# Même résultat en compréhension
cubes = [k ** 3 for k in range(1, 6)]
print(cubes)   # [1, 8, 27, 64, 125]
```

On lit de gauche à droite : « le cube de k, pour k allant de 1 à 5 ». L'ordre d'écriture suit l'ordre de lecture du résultat.

Compréhension avec condition

```python
notes = [12, 15, 9, 18, 8, 14]

admis = [n for n in notes if n >= 10]
print(admis)          # [12, 15, 18, 14]

recales = [n for n in notes if n < 10]
print(recales)        # [9, 8]
```

Le ‘if‘ en fin de compréhension filtre : seuls les éléments qui passent le test entrent dans la liste.

Transformer et filtrer en même temps

```
prix_ht = [100, 250, 40, 999]

prix_ttc = [round(p * 1.18, 2) for p in prix_ht]
print(prix_ttc)

gros_montants = [p * 1.18 for p in prix_ht if p >= 100]
print(gros_montants)
```

L'expression de gauche peut calculer (arrondi, conversion) pendant que le if de droite filtre : les deux rôles sont séparés.

Compréhensions imbriquées

```
matrice = [[1, 2, 3],
           [4, 5, 6]]
ligne_plate = [v for ligne in matrice for v in ligne]
print(ligne_plate)    # [1, 2, 3, 4, 5, 6]

carrees = [(x, y) for x in range(3) for y in range(3)]
print(len(carrees))   # 9
```

Deux for se lisent comme deux boucles imbriquées : le for de droite est la boucle interne. L'ordre des clauses est celui des boucles.

Le piège de la copie

- ‘b = a‘ ne copie **pas** la liste : les deux noms pointent vers le même objet
- Modifier b modifie a : surprise garantie
- Copie superficielle : ‘b = a.copy()‘ ou ‘b = list(a)‘ ou ‘b = a[:]‘
- Vérification : ‘a is b‘ dit si les deux noms désignent le même objet

```
a = [1, 2, 3]
b = a                  # même objet !
b.append(4)
print(a)               # [1, 2, 3, 4]

c = a.copy()           # vraie copie
c.append(99)
print(a, c)
print(a is b, a is c)
```

Attention

IndexError et slices

‘notes[10]‘ sur une liste de 4 éléments lève ‘IndexError‘ : l’index doit être strictement inférieur à la longueur. Étrangement, le slice ‘notes[10:20]‘ ne lève aucune erreur et retourne une liste vide. Rappel : ‘range(len(notes))‘ donne les index valides, mais le parcours direct ‘for n in notes‘ reste plus sûr.

À retenir

L'essentiel de la séance 05

Une liste se parcourt directement et se modifie sur place avec append, insert, remove et pop. Le tuple fige des valeurs liées et sert à retourner plusieurs résultats. La compréhension de liste résume transformation et filtrage en une ligne : expression, for, condition. Et souvenez vous : affecter une liste à un second nom ne la copie pas.

Travaux pratiques (1 h) : Listes, tuples et compréhensions

Du panier de courses au nettoyage de données imbriquées : cinq exercices pour faire de la compréhension de liste un réflexe. Pour les exercices 2 à 5, une solution en une ligne de compréhension est attendue quand c'est possible.

- **Exercice 1 (Basique)** : Panier de courses
- **Exercice 2 (Basique)** : Carrés et cubes en compréhension
- **Exercice 3 (Intermédiaire)** : Filtrer les notes admises
- **Exercice 4 (Intermédiaire)** : Transposée d'une matrice
- **Exercice 5 (Défi)** : Nettoyage de données imbriquées

Ouvrez le notebook « Seance_05_listes_tuples_comprehensions.ipynb » pour commencer.

Pour la prochaine séance

Conseil de travail

Séance 06 : dictionnaires et ensembles pour organiser les données par clé, puis les fonctions pour organiser le code. Relisez le piège de la copie : il réapparaît avec les dictionnaires, exactement de la même façon.